

Agenda

1. Introduction to Recursion
2. Function Call Tracing
3. Factorial of N
4. Printing numbers in Inc Order
5. Printing numbers in Dec Order
6. Time & Space Complexity - factorial, inc/dec
7. Fibonacci

Recursion :- When a function calls itself, it is called as recursion.

Why? A problem when broken down into smaller subproblem & the solution is generated using the subproblems.

Ex:- Cal. the sum of first N natural numbers.

$$N = 5.$$

$$\begin{aligned} \text{Sum}(5) &= 1 + 2 + 3 + 4 + 5 \\ &= \text{Sum}(4) + 5 \end{aligned}$$

↑
Subproblem

$$\text{Sum}(N) = \text{Sum}(N-1) + N^{\text{th}}$$

↑
Recursive relationship

Steps to Write Recursive Code

1. Assumption :- Decide what is the expectation from the function and its inputs & outputs.
2. Main Logic :- Break the problem into smaller subproblems to solve the assumption. / **Recursive Relationship**
3. Identify the base condition :- Identify the inputs for which the recursive function should stop.

```
int Sum (int N) {  
    if (N == 1) { return 1; }  
    return N + Sum (N-1);  
}
```

3

Function Call Tracing

- Function call tracing involves tracing the sequence of function calls that are made when a program is executed.
- It involves keeping track of the function calls, their arguments, and the return values in order to understand the flow of the program and how the functions are interacting with each other.

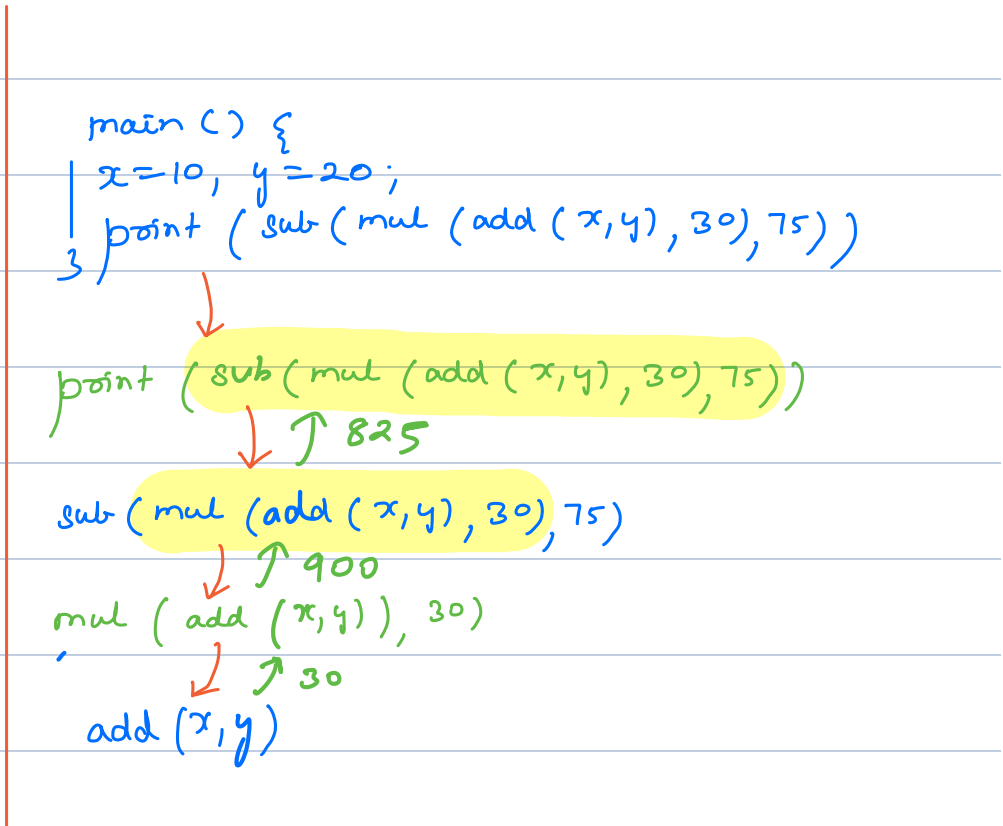
```
function add(x, y) {
    return x + y;
}

function mul(x, y) {
    return x * y;
}

function sub( x, y) {
    return x - y;
}

printfunction ( x) {
    cout << x << endl;
}

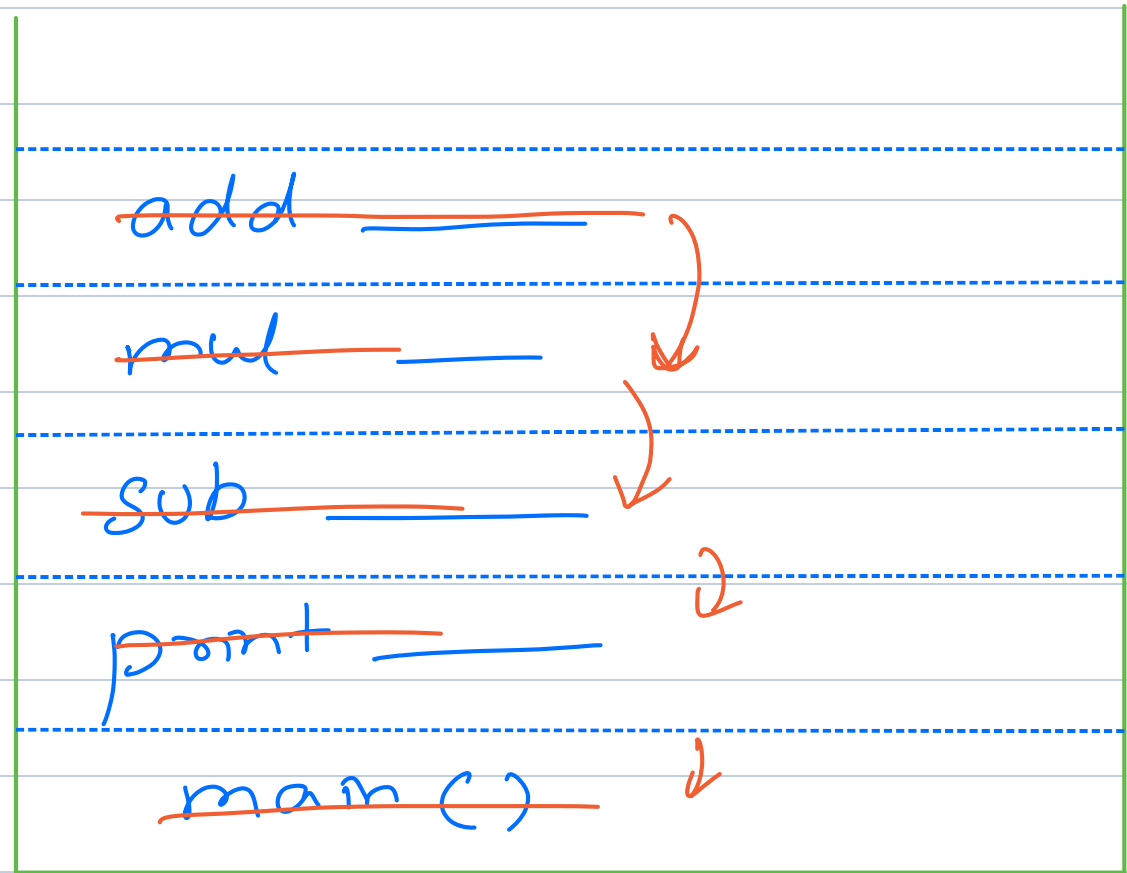
function main() {
    x = 10, y = 20;
    print(sub(mul(add(x, y), 30), 75));
}
```



o/p:- 825:

DS used for function calls

Stack.



o/p = 825

Q:-1. Given N. Print factorial of N.

ex:- $N = 3$

$$3 \times 2 \times 1$$

$$\textcircled{6}$$

$$N = 4$$

$$4 \times 3 \times 2 \times 1$$

$$\textcircled{24}$$

1. $N = 5$

$$5 \times 4 \times 3 \times 2 \times 1$$

$$= 120$$

$$\text{fact}(N) = N \times \text{fact}(N-1);$$

Assumption:- Create a function that takes an integer as the input & return the factorial of it as the output.

Main Logic:-

$$\text{fact}(N) = \text{fact}(N-1) \times N$$

Base Case:-

$$\text{if } (N == 1) \{ \text{return } 1; \}$$

Code

```
int fact ( int N ) {  
    if ( N == 1 ) { return 1; }  
    return N * fact ( N - 1 );  
}
```

T.C $\rightarrow O(N)$

S.C $\rightarrow O(1)$

Tracing

point (fact(3))

3 6

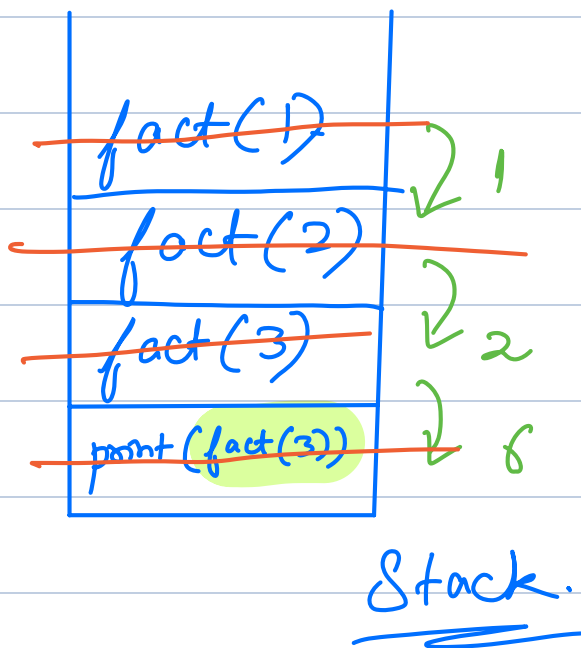
```
int fact ( int N ) {  
    if ( N == 1 ) { return N; }  
    return fact ( N - 1 ) * N;  
}
```

2

```
int fact ( int N ) {  
    if ( N == 1 ) { return N; }  
    return fact ( N - 1 ) * N;  
}
```

1

```
int fact ( int N ) {  
    if ( N == 1 ) { return N; }  
    return fact ( N - 1 ) * N;  
}
```



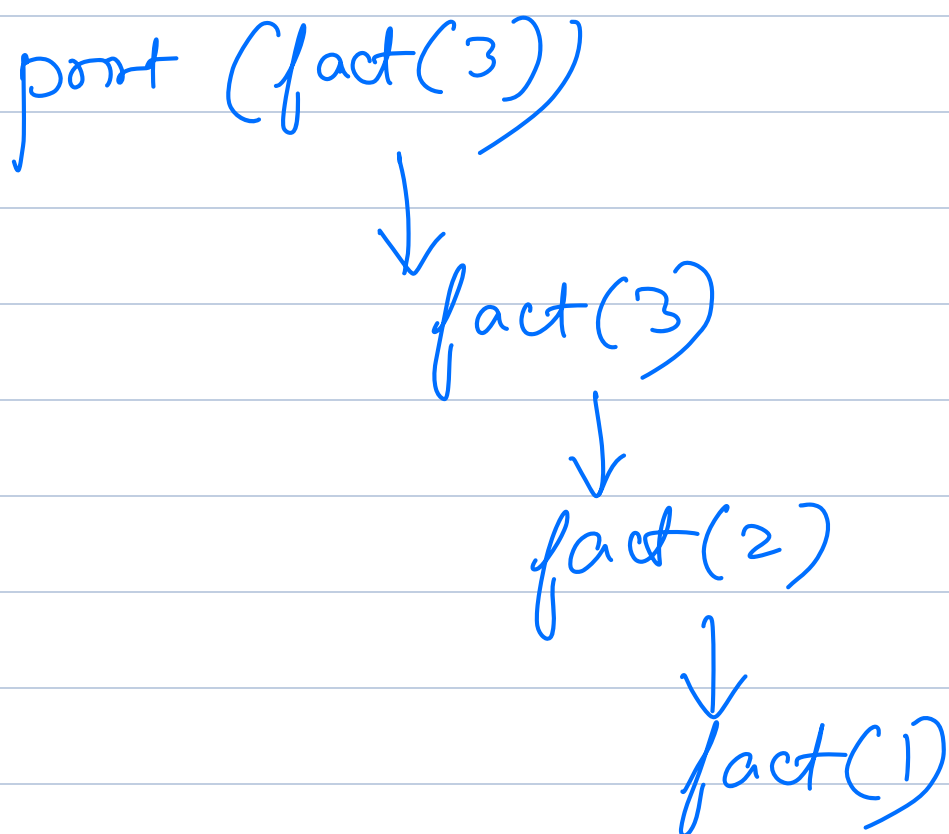
o/p :- 6.

$k \times 3.$

Time & Space Complexity of Recursion

1. T.C. $\rightarrow$ Total T.C. of all the function calls combined.

2. S.C. $\rightarrow$ Height of the recursive tree / recursive stack
(along with any data structure if might be used).



(maximum stack frame at any point of time)

Q1 Print numbers in increasing order.

Ex: $N = 5$

$\text{Inc}(5) = 1 \ 2 \ 3 \ 4 \ 5$
 $= \text{Inc}(4)$
 $\text{print}(5)^{\text{th}}$

Assumption:- Create a function that takes N as the input & print the no.s from 1 to N (Return type $\rightarrow$ void)

Main logic:-

$\text{Inc}(N-1)$
 $\text{print}(N);$

Base Case:-

$\text{if } (N == 1) \{ \text{print}(1);$
 $\text{return}; \}$

Code

void

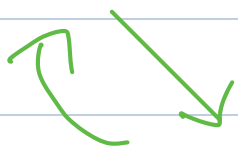
inc (int N) {

if (N == 1) { print (1);
return; }

inc (N-1);
print (N);

3

inc (3)



void inc (int N) {

if (N == 1) {
print ("1");
return;
}

inc (N-1);

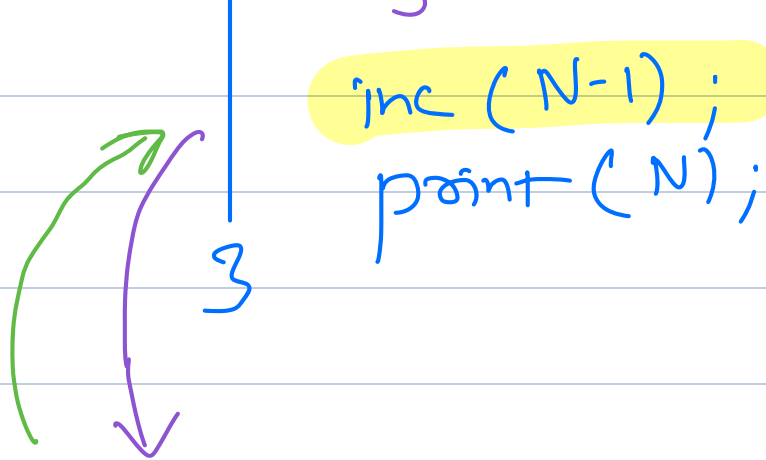
print (N);



void inc (int N) {

if (N == 1) {
print ("1");
return;
}

2



void inc (int N) {

o/p:- 1 2 3

```

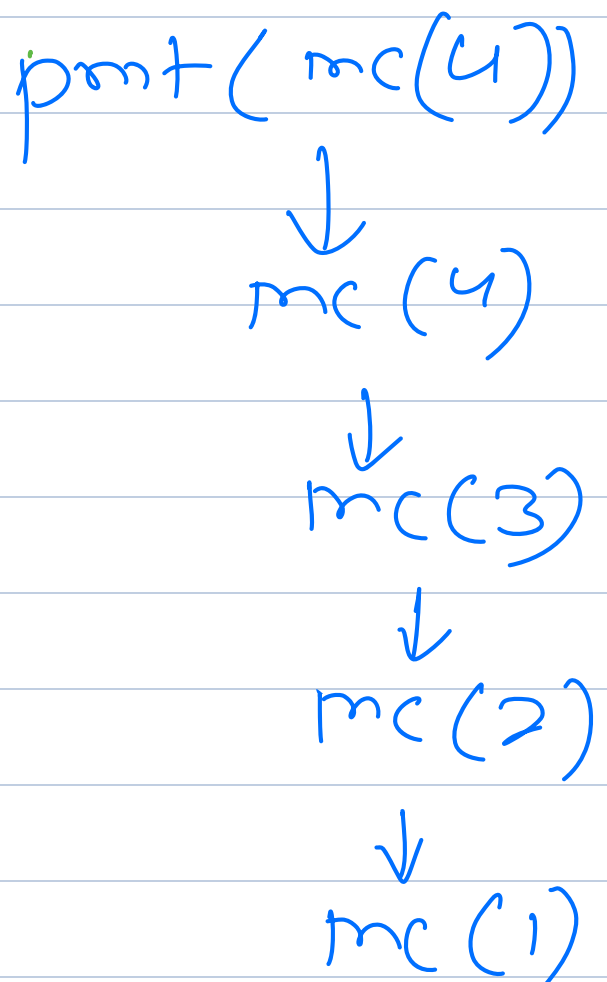
    if (N == 1) {
        print("1");
        return;
    }
    inc(N-1);
    print(N);
}

```

T.C $\rightarrow O(N)$

S.C $\rightarrow O(N)$

8:30



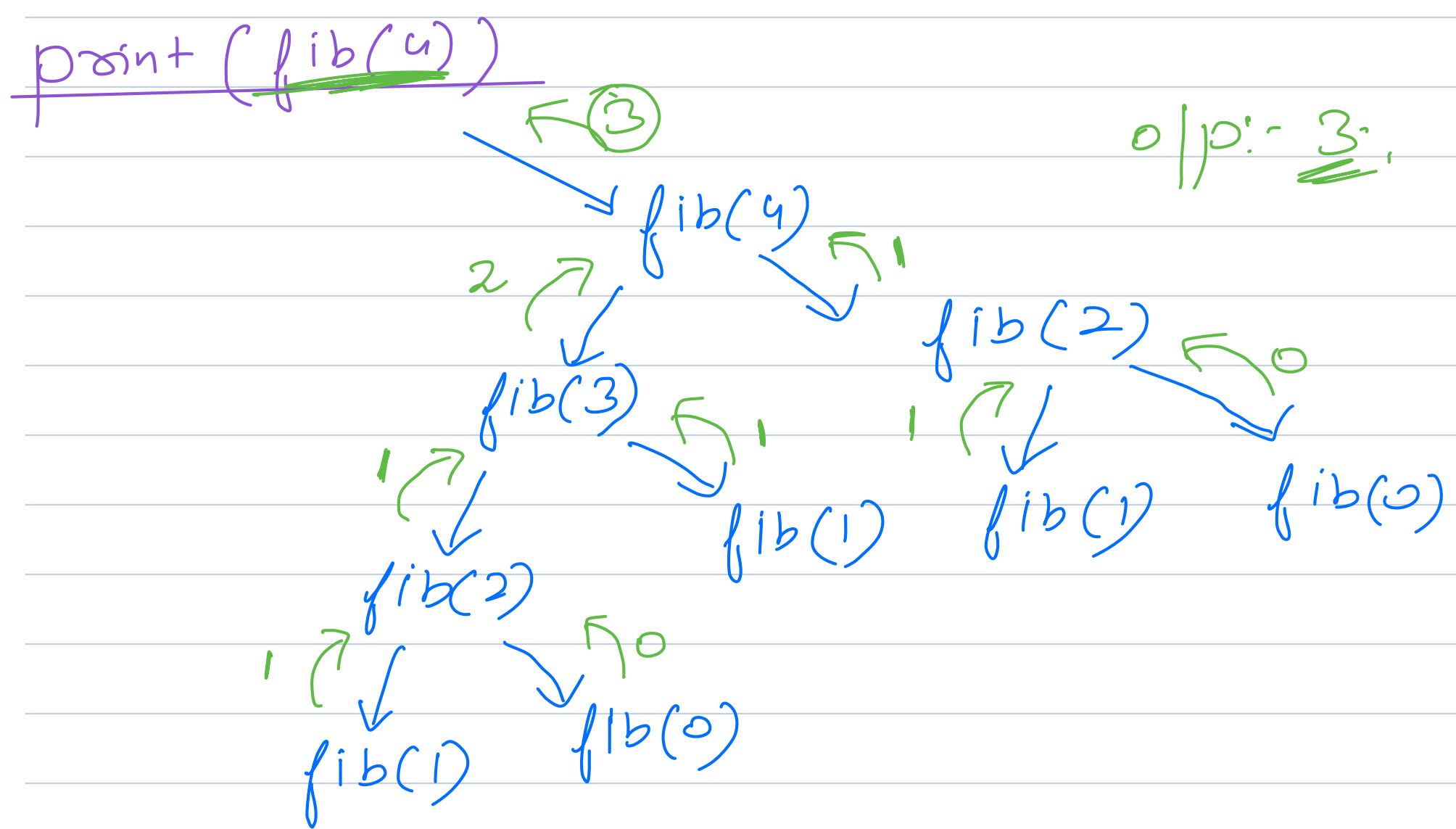
Q:- Print N^{th} no. in fibonacci series

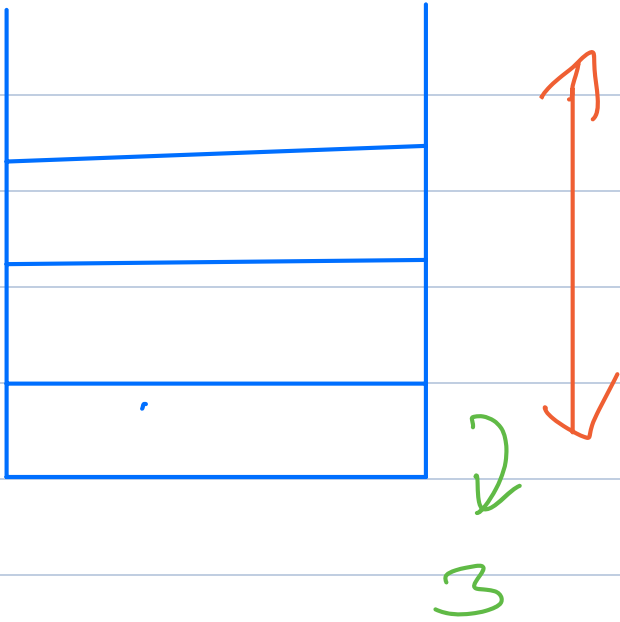
ex:- $N=5$

0	1	2	3	4	5	6	7
0	1	1	2	3	5	8	13

$$fib(N) = fib(N-1) + fib(N-2);$$

```
int fib(int N) {
    if (N <= 1) return N;
    return fib(N-1) + fib(N-2);
}
```

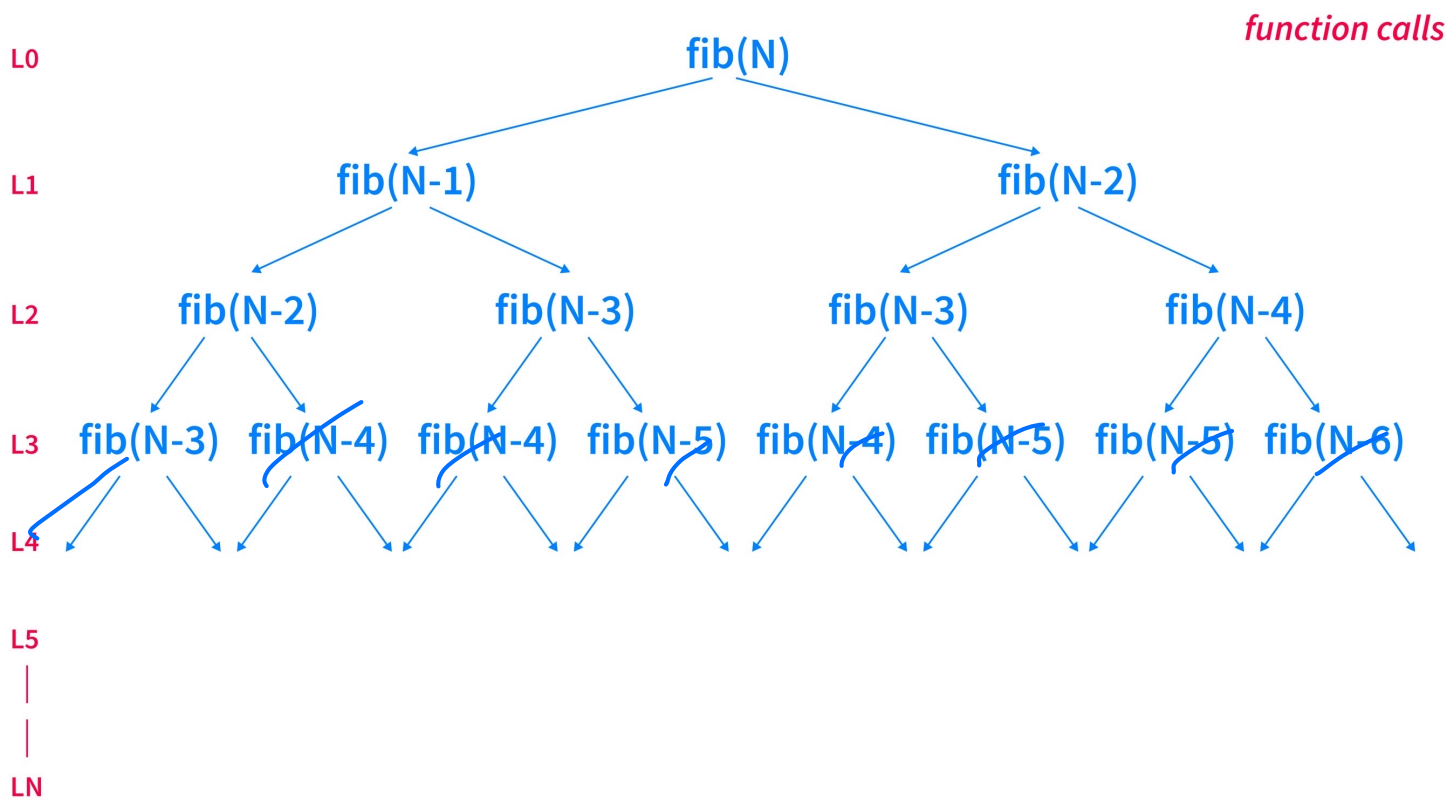




o/p:- 3.

Sc. $O(N)$

T.C. $\rightarrow K \times \text{Total No. of function calls.}$



$$\begin{aligned} 1 &= 2^0 \\ 2 &= 2^1 \\ 4 &= 2^2 \\ 8 &= 2^3 \\ &\vdots \\ N &= 2^N \end{aligned}$$

total function calls :

$$\begin{aligned} \therefore \text{Total function calls} &= 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^N \dots \\ &= a(2^N - 1) \end{aligned}$$

$$8-1$$

$$= 2^0 \frac{(2^{N+1} - 1)}{2-1}$$

$$= 2^{N+1} - 1$$

$$= 2 \cdot 2^N - 1$$

$$T.C = O(2^N).$$

Ans:

Print an array recursively.

ans: $\{ \underset{0}{1}, \underset{1}{2}, \underset{2}{3}, \underset{3}{4}, \underset{4}{5} \}$.

void

print Array (int A[], int i, int N)

if (i == N) { return; }

print (arr[i]);
print Array (A, i+1, N);

3

Dry Run $\rightarrow$ H.W.

